

✓ Generative AI = Powered by FM | LLM

↳ create something new - Content, code, Text generation, Text Summarization

"FM | LLM" = "Pretrained Models" on a very large scale of Data (Internet scale of data)
NLU
 [Raw Data (JSON, Text, PDF, Image, video)]
 ↳ "General Domain"

Providers :

- ✓ Open AI → GPT models
- ✓ Amazon → Nova, Titan
- ✓ Anthropic → Claude
- ✓ AI21 → Jamba
- ✓ Google → Gemini
- ✓ Meta → Llama

Reasoning & Act
 ↓
 Agents

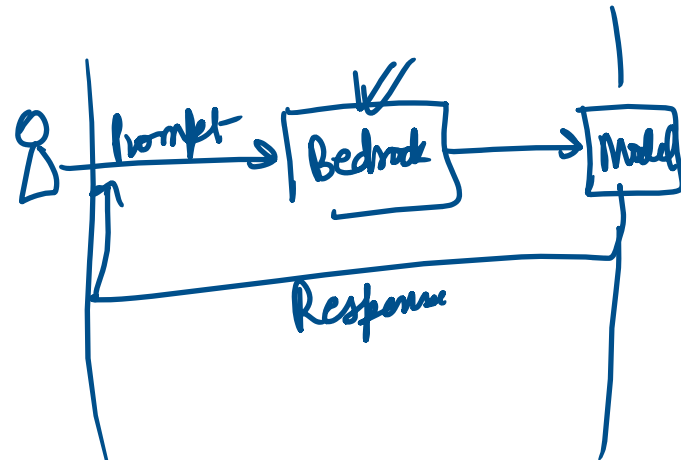
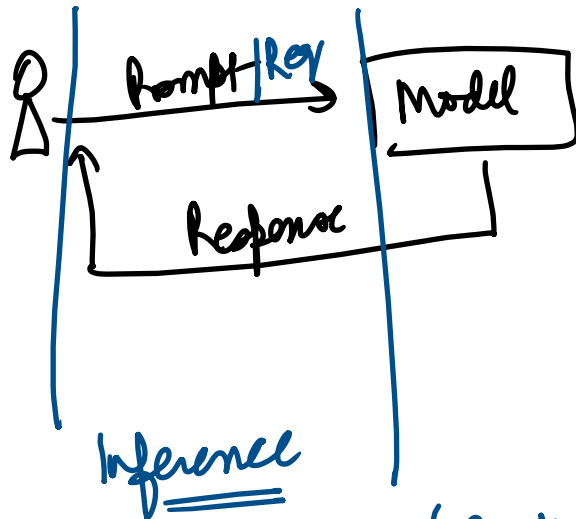
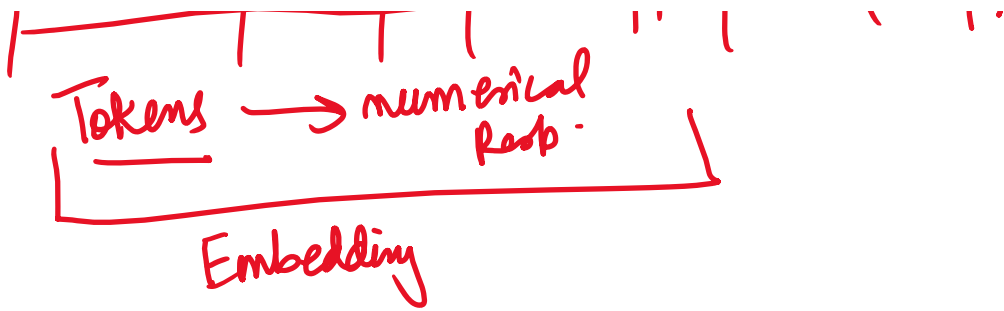
Embedding

RAG

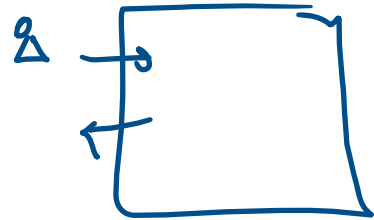
Q Prompt → Numerical Rep (vectors)

Translate	this	into	French:	1	love	cats	.
-----------	------	------	---------	---	------	------	---

→ numerical



- * Inference Parameters (Configurations)
- * Inference profile (Default / Custom)

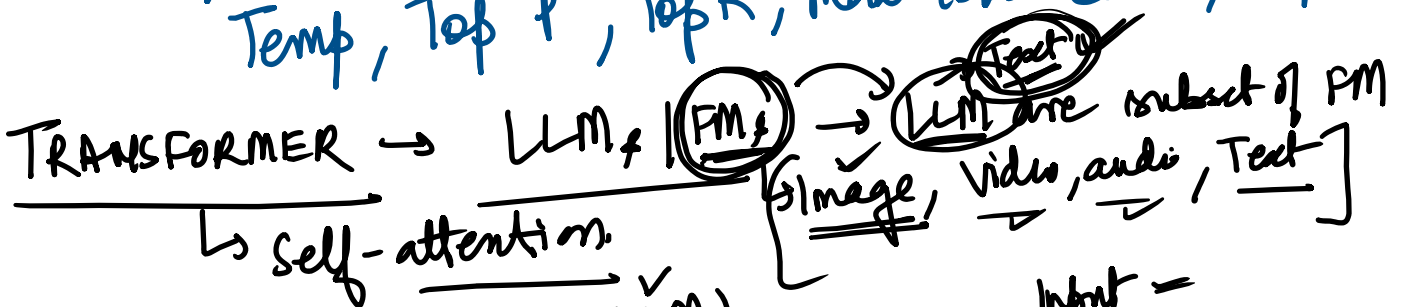


Amazon Bedrock → Model access

How to "control the behavior" of the Model?

Ans: Inference Parameters.

Temp, Top 1, Top K, Max Token count, stop seq.



Input =
Output = Text
"NLP" → Text Generation



Model 1 (FM) → Text ^{mes} ~~transformation~~ ^{into} Text → LM

- ① Tokens → Text is split in Tokens.
- ② Embeddings → Tokens are converted into numerical Rep (vectors)
- ③ Positional Information = Model keeps Tracks of tokens order
- ④ Self-attention ^{neural networks} → each token evaluates with other tokens are relevant.
- ⑤ Feed forward layers = further transform the learned representation.
- ⑥ output

0124
|TRANSLATE| this | in | french | - | I | love | cats |.



"Model" → FM / LM

"Inference parameters / Configurations"

↳ influence how the next token generated by model

① Temperature = Randomness / creativity. $\frac{(0 - 1)}{\text{High} = (0 - 2)}$

0 ↳ Low = Highly focused

0.2 - 0.4 → controlled, factual.

* 0.5 - 0.7 → Balanced

1 ↳ High = Creative / diverse

↳ How random should token selection?

↳ How much probability mass? limits token selection to

Enterprise use case
① Model to Summarize Policy (PDF)

Temp = 0.1

② MKTG content

Temp = 0.8

② Top-P = limits token selection to the smallest group of tokens.

Temp = 0.8

Top-P = 0.90 = 90% [Dog-Puppy, Cat = "_____"]

word	1	50
	2	30
	3	10
	4	5
	5	5

0.40 — 2,3 / 2,4,5

0.20 → 3,4,5

Top P = low = narrower choice
higher = greater

Temp = 1, Top-P = 0.3

③ Top k = Restrict selection to K most probable tokens

Top-k = 3] 3 most likely candidates

How many candidate tokens should be considered?

Analogy = Restaurant menu

"Model has RANKED 100 dishes."

Top-k = 5

[Only look at 5 Highest Ranked dishes]

Top-P = 0.9 (90%)

look at enough dishes to cover 90% of your total.

Temp

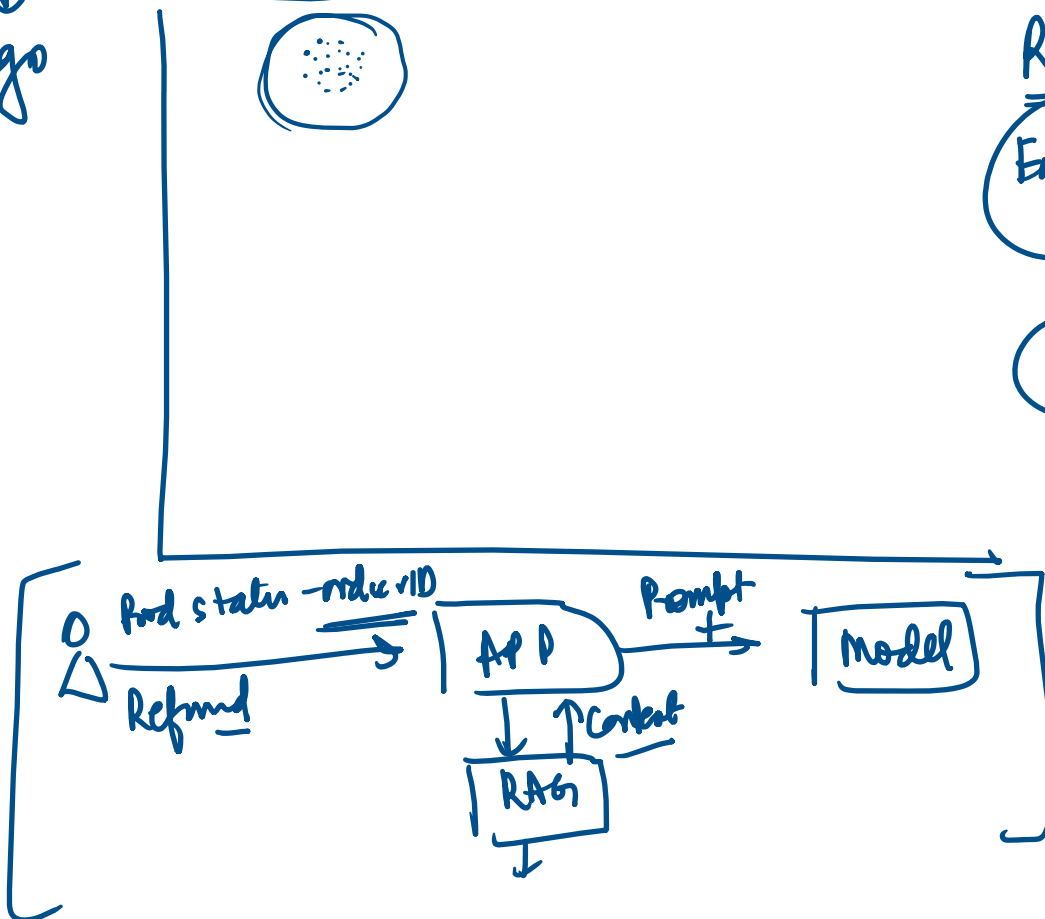
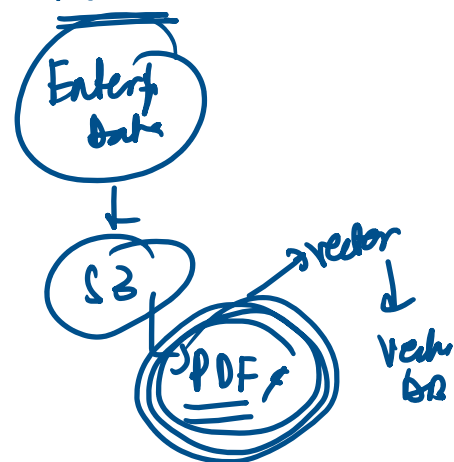
Temp

After creating the shortlist, How willing are you to choose something other than your fav?



[A → B → C]

RAG → vector store



Prompting = what we ask?

Sys Instructions = Influence behavior (Persons, Rules)

Context = what the model knows for this interaction
(Prompt Caching, Prompt History, Conversation History)

Inference Parameters = How the model generates the AI's?

```

system = [
  {"text": "You are a mortgage policy assistant."}
]

messages = [
  {
    "role": "user",
    "content": [
      {"text": "Explain the underwriting policy."}
    ]
  }
]

```

Module 3

Q "PROMPT" engineering?

PROMPT Techniques

optimize prompts

Possible use cases: (Fannie Mae)

- ① Summarizing lengthy policy / Tah Doc
- ② Clarifying service rep.
- ③ Draft explanation from structured info.

Prompt = Instructions. → Model. → Response

↓
Q. "Summarize the doc" ← Input

What? How? Info shouldn't include, Audience.

- ① Boost the model ability

App → "System Instruction / System Prompt" = Persons, Rules.

Common elements of a prompt - [Dev-support Art]

① Instructions = what do I want the model to do?

① Instructions = what do I want the model to do?

System Prompt

User Prompt

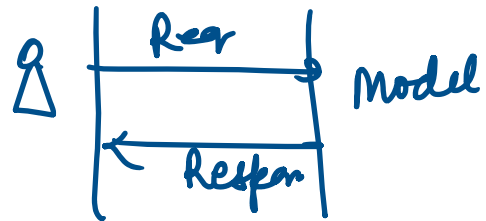
[Analyze the app error & recommend the steps]

② Context = what background info does the model use?
You're Java dev + Prod API dev + Rules
↳ Prompt Caching / Prompt History

③ Input Data = what info should model process?
[Attach pdf] [App log, error ---]
[Attach policy] + user details.

④ Output = what should the results look like?
[Cause, evidences, 3 steps]

Module 4



"Inference" Profile ?

Amazon Bedrock = Configuration → let you control how & WHERE your model requests are processed — especially across regions.

→ Cross-region Inference (Req, Response), Load Balancing across region, HA, Better latency management

Why?

modelID = "amazon.titan-text-express-v1"

Q = global, one region is down, Automatic failover.

λ = global, one region is down, ...
AWS to route to the best region

What?

= Logical wrapper around one/more deployment across AWS Regions.

modelID = amazon.titan-text

- us.
- global.
- eu.
- ap.
- ap.

AWS Decides:

- ① Which region to use?
- ② Where capacity is available
- ③ How to mini latency?

Flows

① Application calls Inference Profile ID

modelID = us.amazon.titan-text

② AWS evaluates

- ① Region Health
- ② Model Capacity
- ③ Latency

③ Request is routed to optimal region

④ Response is returned

Types of Inference Profile

① Cross Region (System-defined)

.us

② Application Inference Profile (Your own Profile)

- ✓ Track usage separately
- ✓ Apply Cost Allocation tags (logs)
- ✓ Control counting preferences.

Amazon Bedrock - Inference Mode

- ① Batch Inference ✓
- ② on-demand → Pay as go
- ③ Provisioned → upfront

* Batch

- ✓ Latency: High (min to Hrs)
- ✓ Cost = More efficient for large jobs
- ✓ Use Case
 - AI reports
 - Predictive Anal.
 - offline processing
 - Large Doc summarizing
 - Nightly jobs

On-Demand

- ✓ Latency: Low (ms to sec)
- ✓ Cost High
- ✓ Use Case
 - AI Agent
 - chatbots
 - [- RAG applications]
 - Code Assist
 - Real-time stream.

Provisioned

- ✓ Latency = Low (ms to sec)
- ✓ Cost: Model units
- ✓ Use Case
 - Enterprise prod sys.
 - High traffic chat/Agent
 - THROUGHPUT